



Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat

Department of Artificial Intelligence

Intelligent Multi-Agent and Expert Systems (AI401), Semester VII

U23AI052 Lab 5 Submission

Experiment 5

Uncertainty Reasoning — Inference in Bayesian Networks

Aim

To specify a Bayesian network as a directed acyclic graph with a conditional probability table at each node, to evaluate entries of the full joint distribution from that factorisation, and to compute the posterior distribution of a query variable given evidence by enumeration.

Theory

The inference rules of Experiment 4 require premises that are certainly true; diagnostic knowledge rarely is. A Bayesian network is a directed acyclic graph whose nodes are random variables and whose edges denote direct dependence, with each node carrying the distribution of the variable conditioned on its parents. By the chain rule the full joint distribution factorises as the product of these conditional tables, so an entry of the joint is one multiplication per node and the network is specified by the table parameters instead of the $2^n - 1$ numbers a joint over n binary variables would need.

Inference means computing the posterior of a query variable X given evidence e , obtained by summing the joint over every assignment of the remaining hidden variables and normalising; the normalising constant is the probability of the evidence itself. The graph also encodes conditional independence: a variable is independent of its non-descendants given its parents, so observing an intermediate node blocks the influence of the variables behind it. The converse pattern is explaining away — two causes that are independent a priori become dependent once their common effect is observed, so confirming one cause lowers the posterior of the other. No sound rule of Experiment 4 can express this, because a derived conclusion there is never weakened by new evidence.

Algorithm

Algorithm 1 — Specifying the network and evaluating a joint entry

1. List the variables in a topological order, and for each variable record its parents and the conditional probability table giving the probability of the variable being true for every combination of parent values.
2. Validate the specification: check that each table has one row per combination of parent values, that every row lies in the unit interval, and count the parameters of the network against the $2^n - 1$ of the full joint.
3. Evaluate a full joint entry for a complete assignment of all variables as the product over nodes of the table entry selected by that variable and its parents, complementing the entry where the variable is false.
4. Verify the factorisation by summing the joint over all 2^n assignments and confirming that the total is one, and obtain the marginal of any single variable by summing over the rest.

Algorithm 1

Parameters: 10 of 31

Joint entry: 0.00025329240000000004

Sum of full joint: 0.9999999999999999

Algorithm 2 — Posterior inference by enumeration with evidence

1. Fix the query variable X and the evidence assignment e , and collect the remaining variables as the hidden set over which the joint will be summed.
2. For each value of X , sum the joint over every assignment of the hidden variables while holding X and e fixed; the two sums form an unnormalised vector whose components are joint probabilities.
3. Normalise the vector to obtain the posterior distribution of X given e , and record the normalising constant separately, since it is the probability of the evidence.
4. Repeat the query with an extra evidence variable added and compare the two posteriors, noting whether the belief in X rises, falls or is unchanged, and relate the direction of change to the paths in the graph.

```
P(Burglary | JohnCalls): {False: 0.98371627005323, True:
0.016283729946769937} P(evidence): 0.0521389757
```

Procedure and Result

Paste the snapshots of the steps followed and the output produced by the program below, including the network structure with its conditional probability tables, the parameter count against the full joint, one evaluated joint entry with the sum over all assignments, and the posterior of the query variable with and without the additional evidence.

Observations

Comment on how many parameters the network saves over the full joint and where that saving comes from, on what the normalising constant of Algorithm 2 measures, on the query whose posterior is unchanged when further evidence is added and on the independence in the graph that explains it, and on why the cost of enumeration grows exponentially with the number of hidden variables.

Exercises

1. Recompute the posterior of the query variable after adding, as evidence, a second cause of the same observed effect. Report both posteriors and the direction of the change, and explain how two variables that are independent when nothing is observed can become dependent once their common effect is known.

```
Exercise 1
P(Burglary | Alarm): {False: 0.6264487717181639, True:
0.37355122828183607}
P(Burglary | Alarm, Earthquake): {False: 0.996731576412303, True:
0.0032684235876969656}
Direction: decreases
```

2. Extend the network with one new node whose single parent is an existing variable, give it a conditional probability table, and recompute the same posterior for both values of the new node. State the new parameter count, then identify one conditional independence that the extended graph asserts and one pair of variables that remains dependent, verifying each numerically.

```
Exercise 2
Parameters: 12 of 63
P(Burglary | JohnCalls, PoliceCalls=False): {False: 0.9882059570121886, True: 0.011794042987811467}
P(Burglary | JohnCalls, PoliceCalls=True): {False: 0.8486123719207512, True: 0.15138762807924872}
Independent check P(Burglary | Alarm), P(Burglary | Alarm, PoliceCalls=False): {False: 0.
6264487717181639, True: 0.3735512282818361} {False: 0.6264487717181639, True: 0.3735512282818361}
Dependent pair check P(Burglary | JohnCalls), P(Burglary | JohnCalls, PoliceCalls=False): {False: 0.
98371627005323, True: 0.016283729946769937} {False: 0.9882059570121886, True: 0.011794042987811467}
```